One rickshaw, 10 styles
the recipes behind every look

The reel is **one** 3D auto-rickshaw built from 24 boxes, cylinders and a sphere in three.js. No Blender, no 3D files, no stock. Every look is the same frame with a different material and a small 2D pass on top. Spin it yourself and tap through the styles at **thenispace.com/styles**. Below: what each style is, how it is done, and the one-line brief to hand Claude Code.

THE BASE: A RICKSHAW FROM PRIMITIVES

- **Shape list, not a model.** Lower shell = a box 1.3 × 0.56 × 2.3. Round nose = a vertical cylinder (radius 0.64) at the front. **One** front wheel under the nose, two at the back. A slanted windshield (a thin box tilted 17°), six thin pillars, a yellow canopy, a bench and seat back, a handlebar, a single round headlight, a number plate.
- **Colours = Delhi auto.** Green body `#157a3a`, yellow canopy `#f2b21b`, dark seats, grey hubs.
- **Camera.** Orbit at radius 13, height 2.6, field of view 30°. Every style starts from the same 3/4-front angle (the one everyone recognises) and turns 22° in 1.8 s.
- **Why primitives.** A list of `{kind, centre, size, colour}` is what lets the voxel and ASCII styles re-sample the same shape instead of needing a new asset.

---

01 WIREFRAME

What: only the edges, cyan on black, a faint 60 px grid behind.

How: `EdgesGeometry(geometry, 18°)` + `LineSegments` in `#19e6ff`; the solid mesh is fully transparent so nothing hides the back edges.

Brief: "Show my three.js scene as a wireframe: EdgesGeometry with an 18° threshold, cyan lines, transparent faces, a thin grid on a near-black background."

02 NEON

What: magenta tube-light outline with a soft glow.

How: the same edge lines in `#ff2fd6`, drawn three times onto a 2D canvas: blurred 16 px at 95 % opacity, blurred 5 px, then sharp with ±1 px offsets so the core line is 3 px wide.

Brief: "Make the wireframe look like neon: magenta lines, draw the frame twice blurred (16 px, 5 px) under a sharp pass, black background."

03 CARTOON

What: flat cel-shaded colours with a black ink outline on a yellow card.

How: `MeshToonMaterial` with a 4-step gradient map (values 40 / 120 / 220 / 255). The outline is a second copy of every mesh in black `MeshBasicMaterial`

with `side: BackSide`, scaled 1.045. Background `#ffd166`.

Brief: "Cel-shade the scene: MeshToonMaterial with a 4-step gradient, back-face black shells scaled 1.045 for the ink line, flat yellow background."

04 CLAY

What: soft matte "stop-motion" model on a pale table.

How: `MeshStandardMaterial`, roughness 0.95, every colour moved 35 % toward white, a stronger hemisphere light, a pale ground disc for the contact-shadow feel. Background `#f4efe8`.

Brief: "Clay render: roughness 0.95, desaturate and lighten the colours 35 %, soft hemisphere light, a light ground disc, warm off-white background."

05 PIXEL ART

What: chunky 8-bit sprite.

How: render the scene at **108 × 192** and scale it up with `imageSmoothingEnabled = false`. `MeshLambertMaterial` gives hard shading steps. Background `#2b2d42`.

Brief: "Pixel-art pass: render at 108×192 and upscale with image smoothing off, Lambert shading, dark navy background."

06 ASCII

What: the rickshaw written in characters, terminal green.

How: render at 80 × 107, read the pixels, map brightness to `" .:-=+*#%@"`, draw bold Menlo characters in three greens on `#020b06`. A gamma of 0.6 keeps the mid-tones visible.

Brief: "ASCII pass: downsample to 80×107, map luminance to the 10-character ramp above, bold monospace glyphs in green on near-black, gamma 0.6."

07 BLUEPRINT

What: white drawing lines on engineering blue with two grids.

How: white edge lines over `#0b3d91`, a 48 px faint grid and a 240 px stronger grid, faces 55 % transparent blue so the far edges show through.

Brief: "Blueprint look: white edges, faces in translucent blue, two overlaid grids (48 px, 240 px) on #0b3d91."

08 VOXEL

What: the rickshaw as about 3,000 small cubes.

How: sample the primitive list on a 0.12 grid with point-in-box / point-in-cylinder tests, then one `InstancedMesh` of 0.11 cubes with a per-instance colour. No new geometry, the shape list does the work.

Brief: "Voxelise my primitive list: test every 0.12 grid point against each box, cylinder and sphere, one InstancedMesh of cubes coloured per hit."

09 PAPER CUT

What: flat paper layers with a drop shadow.

How: `MeshBasicMaterial` colours lightened 20 %, black back-face outline, and a blurred dark copy of the frame drawn 22 px right and 30 px down at 28 % as the shadow. Paper `#f3eee4`.

Brief: "Paper-cut pass: flat lightened colours, thin black outline, a blurred dark copy of the frame offset 22/30 px at 28 % as a drop shadow, paper background."

10 GLITCH

What: the cartoon frame tearing with colour fringes and scanlines.

How: six horizontal slices per frame copied sideways up to ±45 px (positions come from a hash of the frame number, so it is repeatable), red and cyan copies at ±7 px in `screen` mode at 32 %, 2 px scanlines.

Brief: "Glitch pass: 6 hashed horizontal slice shifts per frame, red/cyan channel copies at ±7 px in screen mode, 2 px scanlines."

THE RULES THAT KEPT IT CLEAN

- **One scene, one camera, ten materials.** Never ten models.
- **Same angle every time.** Each look starts at the same 3/4-front angle and turns slowly, so your eye compares styles, not poses.
- **Frame by frame.** Rendered with HyperFrames (`npx hyperframes render --resolution portrait-4k`): everything is a function of time, no randomness, no timers, so every render is identical.
- **Sound under, not over.** A quiet ACE-Step bed (local, royalty-free) and one soft click on every style switch. No whooshes.

TOOLS (ALL FREE)

Claude Code · three.js r160 · a 2D canvas for the passes · HyperFrames for the render · ACE-Step 1.5 for the music

Questions? DM @thenispace. — NIS · AI · Tech · Productivity